
Optimized Modeling and Predictive Analysis of Complex Systems

Summary

Problem Restatement. In one or two sentences, state the **core question** and **constraints** in your own academic language; specify the **decision variables**, **objective**, and **feasible region**.

Uniqueness of the Modeling Approach. Briefly state the **model class** (e.g., differential equations, optimization, stochastic simulation) and the **distinct assumptions** or **innovations** relative to existing methods.

Core Algorithm Logic. Outline the **solution pipeline**: inputs, main iterations or recurrences, and output metrics; when relevant, name the **software/algorithms** used (e.g., Gurobi, Monte Carlo).

Most Impactful Quantitative Conclusions. Provide concrete **numerical results** and **policy implications** (e.g., “cost reduced by 15.4%” or “service level above 98%”); avoid vague phrasing. Optionally state key **sensitivity** or **robustness** findings.

We use bold terms and short paragraphs for modular scanning, with clear spacing between blocks.

Keywords: Mathematical Modeling; Sensitivity Analysis; AI Tools Disclosure.

Contents

1	Introduction	3
1.1	Background and Motivation	3
1.2	Literature Review	3
1.3	Restatement of the Problem	3
2	Assumptions and Notations	4
2.1	General Assumptions	4
2.2	Notation Definition	4
3	Data Sources	6
3.1	Data Description and Task DNA Source	6
4	Model Construction and Implementation	7
4.1	Career Selection as a Modeled Decision	7
4.1.1	Objectifying Career Feature Parameters with O*NET	7
4.1.2	Revision and Final Configuration of Selected Occupations	10
4.2	Model Selection for Occupational Evolution	10
4.2.1	Computable Definition of Future Evolution	10
4.2.2	Comparison of Candidate Modeling Approaches	11
4.2.3	Final Model Choice	11
4.3	Task-Aggregate Dynamic Model of Occupational Evolution	12
4.3.1	Task Aggregation and Occupational Dynamics	12
4.4	GenAI Capability-Dimension Scoring from Authoritative Reports: Scale Definition and Evidence Narrative	12
4.4.1	Scale Definition	12
4.4.2	Capability Evidence by Task Type (Narrative Justification)	13
4.5	Authoritative Evaluation Driven Relabeling of Task Impacts: From Keyword Tagging to Capability-Dimension Assignment	14
4.6	Aggregating Task DNA into Occupation-Level Shock Parameters: Software Developers	16
4.6.1	Top- N Availability and Aggregation	16
4.6.2	Interpretability Decomposition: Top- K Contribution Lower Bounds	16
4.6.3	Validity Check of the Scoring Rule: Avoiding Near-Constant s/c	16
4.6.4	Minimal Calibration: Category Defaults with Limited Manual Review	17
5	Mechanistic Power Check and Occupation-Level Aggregation After Authoritative Relabeling (Software Developer)	17
5.1	Occupation-Level Shock Aggregates	17
5.2	Diagnostics: Heterogeneity and Effective Diversity	17
5.3	Engineering Thresholds and Actions (Gatekeeping Before $E(t)$)	18
6	Consistency Check and Instantiating the Evolution Equation: (α, β, μ) for Software Developers	18
6.1	Aggregates and Consistency Check	18

6.2	Mechanistic Power (Heterogeneity) Check: No Score Collapse	19
6.3	Minimal Sensitivity for a Low-Confidence Task (Upper-Bound Argument)	19
6.4	Proceeding to Occupational Evolution After Fixing μ	19
7	Task-Level Functional Demand Model and Occupational Redefinition	19
7.1	Task-Level Functional Demand Model (Replacing Occupation-Level Job Counts) .	19
7.1.1	Modeling Objective and Notation	19
7.1.2	Two-Channel Dynamics: Automation and Scaling	20
7.1.3	Adoption Scenarios $A(t)$	21
7.1.4	Occupational Redefinition Triggers (Change in the Nature of Work)	21
7.1.5	Computation Algorithm (Reproducible Workflow)	22
7.1.6	Parameter Table (To Be Instantiated)	22
7.2	Visualization Plan	22
7.2.1	Figure Templates	22
7.3	Sensitivity and Robustness	23
7.3.1	Low-Confidence Task Sensitivity	23
7.3.2	Key-Parameter Sensitivity	23
7.4	Output Summary Template (Per Occupation)	24
8	Simulation Results and Sensitivity Analysis	24
9	Discussion: Strengths and Weaknesses	24
9.1	Strengths	24
9.2	Weaknesses	24
10	Conclusion	24
	Appendices	24
	Appendix A Python Implementation Code	24

1 Introduction

1.1 Background and Motivation

Generative artificial intelligence (Gen-AI) is evolving from a niche assistant into foundational infrastructure across industries. As capabilities grow exponentially, Gen-AI not only streamlines workflows but also reshapes the task structure of labor markets. Under this rapid technological shift, higher-education institutions (comprehensive universities, vocational colleges, and arts schools) face an urgent challenge: how to adjust training models so that graduates retain durable competitiveness and adaptability in AI-saturated workplaces.

AI use disclosure. We disclose our AI tool usage in-text and in the **Report on Use of AI Tools** at the end of this paper.

1.2 Literature Review

We cite prior studies to strengthen the credibility of our model.[?]

1.3 Restatement of the Problem

Part I (occupation level: future evolution under Gen-AI impact). The task requires us to select one occupation each from **STEM, Trade, and Arts**, and to characterize how these occupations evolve under widespread Gen-AI adoption using traceable data and citations. Here, “future” should be interpreted as evolution within a clear forecasting window (e.g., the next several years): directions and magnitudes of contraction, stability, or expansion in task content and job demand. The “current trajectory” refers to how Gen-AI penetration and use in the industry are trending, while the “expected impact” refers to measurable changes in job structure and work practices. Each judgment must state the data sources used and explain the key drivers we believe will push occupational change.

Part II (education program level: actionable recommendations for three specific programs). After the occupation-level analysis, the task further requires a “corresponding institution and program” for each occupation (from a university, a vocational/apprenticeship system, and an arts school) and asks us to translate conclusions into decisions for program managers. Recommendations should cover at least two core questions: whether program scale should expand or contract with occupational change (and how to attract more students in expansion or absorb students in contraction), and what the program should teach and how it should handle Gen-AI use (e.g., bans, limited permission, or integration into curricula and training). Here, “best support/best prepare” must mean **actionable and comparable** plans, supported by model outputs rather than slogans.

Part III (evaluation criteria: beyond employability, and limits of extrapolation). The task notes that “graduate employability” may not be the only success metric, so we must propose and justify additional dimensions and explain how conclusions change when those dimensions are included. To avoid ambiguity, “employability” can be understood as graduates’ ability to obtain and retain jobs in an AI-ubiquitous market. “Other factors” may include, but are not limited to, resource costs of teaching and using Gen-AI (energy/water) and attribution or integrity risks in academic and creative settings. Finally, if we believe recommendations for one program can generalize to others, we must specify the conditions and boundaries under which extrapolation holds or fails.

2 Assumptions and Notations

2.1 General Assumptions

2.2 Notation Definition

We define all symbols in the unified table below.

Table 1: Unified Notation Table

Symbol	Description	Domain/Units
Basic Indices		
i	Occupation index	$\mathbb{Z}^+$
j	Task index	$\mathbb{Z}^+$
t	Time	Years
N	Number of usable tasks	$\mathbb{Z}^+$
Career Selection and Feature Space		
x	Candidate occupation	—
$f(x)$	Occupation feature vector ($digit(x)$, $physical(x)$, $iprisk(x)$)	$[0, 1]^3$
$digit(x)$	Digitizability	$[0, 1]$
$physical(x)$	Physical anchoring	$[0, 1]$
$iprisk(x)$	IP/attribution risk	$[0, 1]$
$D(x)$	Data availability score	$[0, 1]$
$\Delta(x_i, x_j)$	Mechanistic contrast distance: $\ f(x_i) - f(x_j)\ _2$	$\mathbb{R}_{\geq 0}$
λ	Trade-off coefficient in selection	Dimensionless
$v_m(x)$	Raw rating for O*NET descriptor m of occupation x	$[0, 100]$
$z_m(x)$	Min-max normalized rating	$[0, 1]$
Task Weights and Impact Parameters		
w_{ij}	Weight of task j in occupation i ; $\sum_j w_{ij} = 1$	$[0, 1]$
$w_{ij}(0)$	Baseline task weight (initial time)	$[0, 1]$
$w_{ij}(t)$	Time-varying task labor share (task structure)	$[0, 1]$
s_{ij}	Substitution score for task j (GenAI reduces labor effort)	$\{0.25, 0.5, 0.75\}$
c_{ij}	Augmentation score for task j (GenAI expands functional demand)	$\{0.25, 0.5, 0.75\}$
α_i	Occupation-level substitution: $\sum_j w_{ij}s_{ij}$	$[0, 1]$
β_i	Occupation-level augmentation: $\sum_j w_{ij}c_{ij}$	$[0, 1]$
μ_i	Net shock: $\beta_i - \alpha_i$	$[-1, 1]$
$\bar{c}_i$	Augmentation potential: $\sum_j w_{ij}(0)c_{ij}$	$[0, 1]$
Occupational Evolution Dynamics		
$E_i(t)$	Employment demand for occupation i at time t	Jobs
$A(t)$	GenAI adoption trajectory (fast/medium/slow scenarios)	$[0, 1]$
g_i	Non-AI baseline growth rate for occupation i	yr^{-1}

Continued

Table 1 – continued

Symbol	Description	Domain/Units
k	Adoption-curve steepness parameter	> 0
t_0	Adoption inflection time (50% adoption)	Years
Task-Level Functional Demand Model		
$Q_i(t)$	Overall functional output demand index for occupation i	Dimensionless
$\ell_{ij}(t)$	Labor intensity of task j (labor per unit output)	Dimensionless
$D_{ij}(t)$	Labor-demand index for task j	Dimensionless
κ_s	Substitution speed coefficient	> 0
κ_c	Augmentation demand multiplier	≥ 0
Occupational Redefinition Triggers		
$M_{\text{drop}}(t)$	Extinction mass (baseline weight of negligible tasks)	$[0, 1]$
ε	“Negligible task” threshold (e.g., 0.05)	$(0, 1)$
θ	Extinction-mass trigger threshold	$(0, 1)$
δ	Task-share drift threshold (Jensen–Shannon)	> 0
t^*	Redefinition trigger time	Years
$\text{JS}(\cdot \parallel \cdot)$	Jensen–Shannon divergence	$[0, 1]$
Mechanistic Power Diagnostics		
$\text{Var}_w(s)$	Weighted variance of substitution: $\sum_j w_j (s_j - \alpha)^2$	$\mathbb{R}_{\geq 0}$
$\text{Var}_w(c)$	Weighted variance of augmentation: $\sum_j w_j (c_j - \beta)^2$	$\mathbb{R}_{\geq 0}$
$h_s(g)$	Weight mass at substitution tier g	$[0, 1]$
$h_c(g)$	Weight mass at augmentation tier g	$[0, 1]$
$m(d)$	Weight mass at capability dimension d	$[0, 1]$
$\varepsilon_s, \varepsilon_c$	Minimum variance thresholds (default 0.005)	$\mathbb{R}_{>0}$
η_s, η_d	Maximum concentration thresholds (default 0.70, 0.60)	$(0, 1)$
Authoritative Relabeling Pipeline		
dim_j	Capability dimensions mapped to task j	Set
d	Capability dimension index (e.g., code generation, debugging)	—
S_d	Substitution tier for dimension d	$\{0.25, 0.5, 0.75\}$
C_d	Augmentation tier for dimension d	$\{0.25, 0.5, 0.75\}$
$\text{sim}(j, d)$	Semantic similarity between task j and dimension d	$[0, 1]$
τ	Semantic matching threshold	$[0, 1]$
π_{jd}	Allocation weight from task j to dimension d	$[0, 1]$
Task DNA Fields (O*NET Data)		
Task_j	Task statement text	Text
IM_j	Task importance rating	$[0, 100]$
FR_j	Task frequency rating	$[0, 100]$
$\tilde{w}_j$	Unnormalized weight: $\text{IM}_j \cdot \text{FR}_j$	$\mathbb{R}_{\geq 0}$
SOC	O*NET-SOC occupation code	Code

3 Data Sources

3.1 Data Description and Task DNA Source

We extract task statements and task ratings for SOC 15-1252.00 from the O*NET 30.1 text database, and we compute task weights from Importance (IM) and Frequency of Task (FT) ratings.[?]

Table 2: Data Sources and Variables for Task DNA

Variable/Data	Description	Source	
O*NET Task Statements	Task text and identifiers for SOC 15-1252.00	O*NET database	30.1
O*NET Task Ratings (IM, FT)	Importance and Frequency ratings used to compute task weights	O*NET database	30.1
Task DNA CSV	Processed task table with IM, FR, weight, and rule-based s/c	Derived from O*NET 30.1	[?]

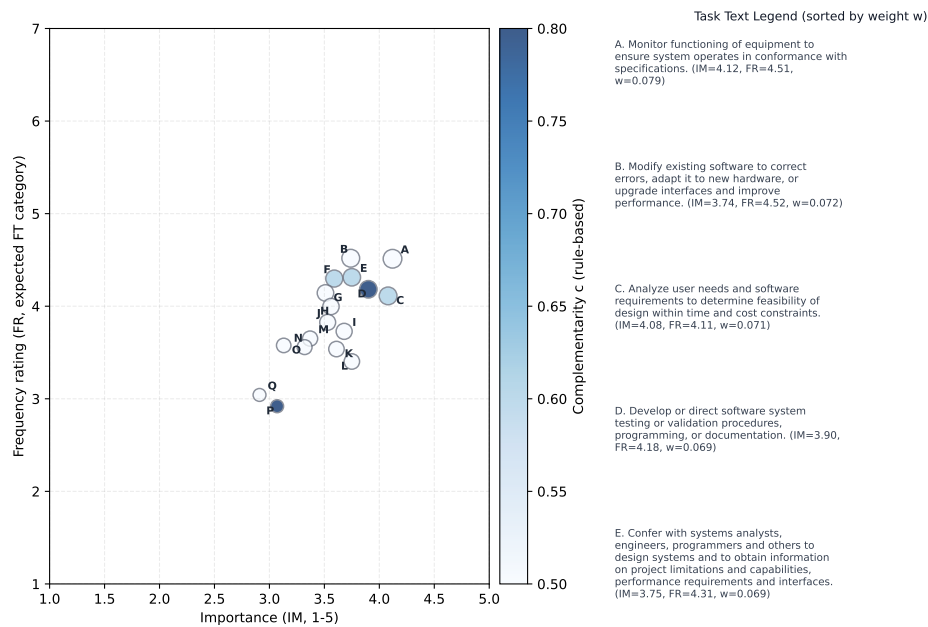


Figure 1: Task-level IM–FR exposure for Software Developers (SOC 15-1252.00). Bubble size encodes normalized task weight w , color encodes rule-based complementarity c , and edge thickness reflects rule-based substitution s . Letter tags map to the task text legend panel.

4 Model Construction and Implementation

4.1 Career Selection as a Modeled Decision

The task requires selecting one occupation each from the STEM, Trade, and Arts sectors. We do not treat this step as an ad hoc assumption; instead, we explicitly model career selection as an optimization problem. This ensures that subsequent analyses of AI-driven impacts and educational recommendations are grounded in a reproducible, data-informed framework. The core objective of career selection is not representativeness of all occupations, but *explanatory contrast*. Specifically, the selected careers should differ substantially in how GenAI interacts with their task structures. Beyond mechanistic contrast, the task emphasizes data-informed reasoning and policy recommendations, so selected careers must be both data-supported and policy-relevant.

4.1.1 Objectifying Career Feature Parameters with O*NET

To avoid subjective assignment of the career feature parameters $digit(x)$, $physical(x)$, and $iprisk(x)$, we adopt the O*NET occupational database maintained by the U.S. Department of Labor as a unified data source; Section 3 summarizes the data provenance. O*NET provides numerous *numerical scale descriptors* for each occupation based on expert surveys and practitioner feedback, with most indicators on a continuous scale of 0–100, suitable for quantitative modeling.

We do not attempt to find a single “correct” indicator for each axis. For each feature axis, we select a small number of semantically direct and downloadable O*NET descriptors and use them as proxy variables.

Based on the task statement and data availability constraints, we identify three dominant dimensions that govern heterogeneity in AI impact:

- **Digitizability** $digit(x)$: the extent to which core tasks can be represented, automated, or augmented digitally;
- **Physical anchoring** $physical(x)$: the degree to which task execution depends on on-site physical presence or manual interaction with the environment;
- **Intellectual property and attribution risk** $iprisk(x)$: the sensitivity of value creation to authorship, originality, and ownership.

Each candidate occupation x is represented by a normalized three-dimensional feature vector

$$f(x) = (digit(x), physical(x), iprisk(x)) \in [0, 1]^3, \quad (1)$$

where higher values indicate stronger presence of the corresponding characteristic.

For three occupations (x_1, x_2, x_3) drawn respectively from STEM, Trade, and Arts categories, we define the contrast between two occupations as the Euclidean distance between their feature vectors:

$$\Delta(x_i, x_j) = \|f(x_i) - f(x_j)\|_2. \quad (2)$$

We define a data availability score $D(x_k)$ to reflect how accessible and rich the data are for occupation x_k . The overall selection objective is

$$\max_{x_1 \in STEM, x_2 \in Trade, x_3 \in Arts} \left(\sum_{i < j} \Delta(x_i, x_j) + \lambda \sum_{k=1}^3 D(x_k) \right), \quad (3)$$

where λ balances explanatory contrast and empirical feasibility.

This formulation does not claim global optimality across all possible occupations. Instead, it yields a deliberately constructed, high-contrast set that maximizes analytical clarity under realistic data and time constraints.

Normalization and Reproducibility For each occupation x , let $v_m(x)$ denote its raw rating (0–100) under O*NET descriptor m . We apply min–max normalization across the entire occupation set:

$$z_m(x) = \frac{v_m(x) - \min_y v_m(y)}{\max_y v_m(y) - \min_y v_m(y)} \in [0, 1], \quad (4)$$

and take the equally weighted average of multiple descriptors under the same feature axis to obtain $digit(x)$, $physical(x)$, and $iprisk(x)$. This process is fully based on publicly available data and explicit rules, ensuring reproducibility and auditability.

We visualize the three-dimensional heterogeneity and its 2D projections to validate that the selected careers occupy distinct regions in the feature space (Fig. 2 and Fig. 3).

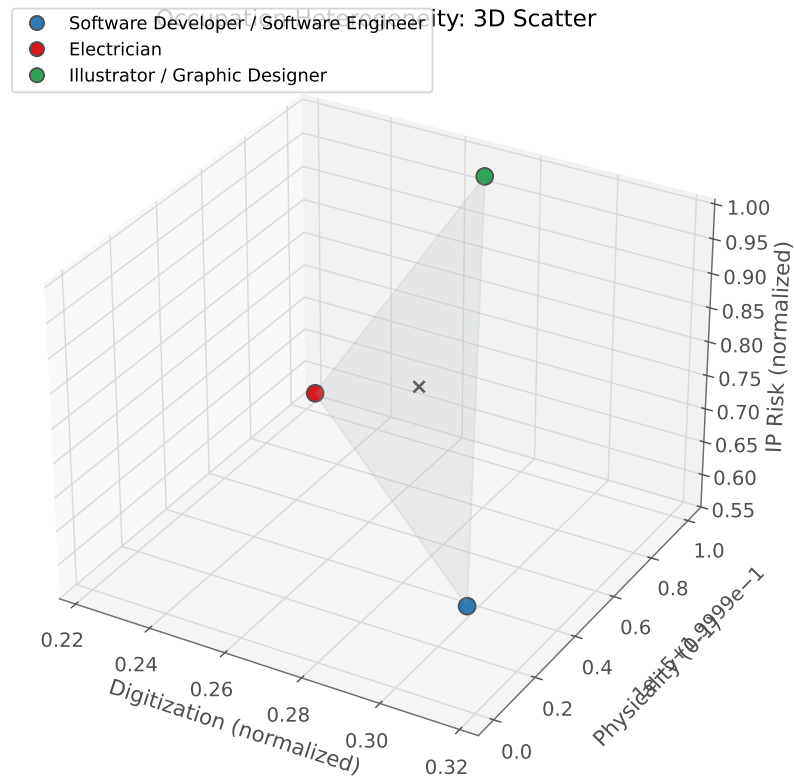


Figure 2: 3D heterogeneity of the selected STEM, Trade, and Arts occupations based on O*NET descriptors (global normalization).

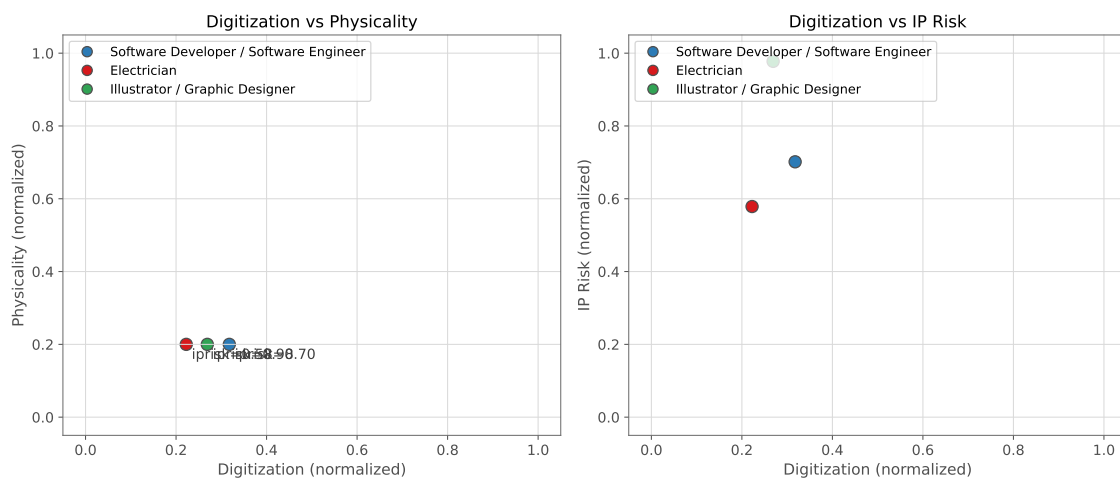


Figure 3: 2D projections of the heterogeneity space; annotations report the remaining axis value for interpretability (global normalization).

4.1.2 Revision and Final Configuration of Selected Occupations

In the initial scheme, we selected a structural/civil engineer as the STEM occupation. However, after examining how the three occupations distribute along the “task structure” and “physical constraint” dimensions, we found that this choice is somewhat similar to the Trade occupation (electrician) along the “physical anchoring strength” axis, which weakens contrast in mechanism space.

To maximize task-structure heterogeneity and improve identifiability of subsequent GenAI impact mechanisms, we revise the STEM endpoint and replace it with **Software Developer / Software Engineer**.

This revision does not alter the structure of the original selection model; it only shifts the endpoint in feature space. Software developers exhibit extremely high digitizability and extremely low physical anchoring, which increases their distance from electricians in $f(x)$ and improves the objective value of the contrast function.

We emphasize that we do not assume software development will be simply “replaced” by GenAI. Instead, this occupation is more likely to undergo **task-structure reorganization**: the share of coding and boilerplate implementation decreases, while tasks such as requirements clarification, system architecture design, verification testing, and security/compliance responsibilities increase. This shift from “execution-oriented tasks” to “verification and responsibility-oriented tasks” makes software development a strong STEM endpoint for studying GenAI mechanisms.

After this revision, the final configuration of the three selected occupations is:

- **STEM: Software Developer / Software Engineer.** High digitizability, low physical anchoring; GenAI can directly intervene in core tasks, but the dominant effect is task redistribution rather than occupational extinction.
- **Trade: Electrician.** High physical anchoring and strong on-site constraints; AI primarily serves as a planning, diagnostic, and auxiliary tool, and cannot replace core operational tasks.
- **Arts: Commercial Illustrator / Graphic Designer.** High IP and attribution sensitivity; GenAI directly affects output forms and value pricing mechanisms.

4.2 Model Selection for Occupational Evolution

The second core requirement is to build a *data-informed* model of future occupational evolution. Before introducing formal equations, we define what “future evolution” means in computable terms and then compare alternative modeling approaches so that model choice is not an implicit assumption.

4.2.1 Computable Definition of Future Evolution

We characterize future evolution of occupation i with two primary outputs:

- **Occupational demand:** time evolution of job demand $E_i(t)$ over a 10-year horizon under fast, medium, and slow GenAI adoption scenarios;

- **Task structure:** reallocation of task weights $T_i(t)$ within the occupation, indicating which tasks expand, contract, or are reorganized.

This definition ensures occupation-level outcomes are derived from task-level mechanisms rather than imposed directly.

4.2.2 Comparison of Candidate Modeling Approaches

We consider three representative approaches:

Approach A: Task-aggregate impact model This approach treats an occupation as a weighted bundle of tasks. GenAI-driven substitution and augmentation are specified at the task level and aggregated to the occupation level using task weights. Most parameters can be constructed from O*NET task data [?] and public AI exposure rubrics [?], providing transparency and reproducibility.

Approach B: Job-posting time-series models This approach infers occupational change from trends in AI-related keywords and skill requirements in job postings. Although close to market signals, it suffers from identification challenges and high data-processing overhead, so it is unsuitable as the main modeling backbone.

Approach C: Agent-based simulations Agent-based models simulate strategic interactions among firms, workers, and educational institutions. Despite narrative richness, they require extensive calibration and introduce substantial model complexity, which conflicts with transparency requirements under strict page limits.

4.2.3 Final Model Choice

Based on the comparison, we adopt the task-aggregate impact model (Approach A) as our primary framework. This choice is motivated by:

- **Mechanistic clarity:** occupational change emerges explicitly from task-level effects;
- **Data alignment:** key parameters are grounded in publicly available occupational data [? ?];
- **Controlled complexity:** the model remains interpretable and suitable for scenario and sensitivity analysis;
- **Transferability:** the framework can be extended to other occupations or educational pathways [?].

We reserve Approach B for potential validation or supplementary analysis, while Approach C is excluded from the main analysis due to calibration and transparency concerns.

4.3 Task-Aggregate Dynamic Model of Occupational Evolution

Having justified the task-aggregate approach, we present its mathematical formulation. We adopt a minimal task-to-occupation framework: GenAI adoption $A(t)$ enters as an exogenous driver, substitution and augmentation are specified at the task level, and the occupation-level net impact is derived by task-weight aggregation, yielding a dynamic trajectory for occupational demand $E_i(t)$.

4.3.1 Task Aggregation and Occupational Dynamics

Let occupation i consist of tasks $j = 1, \dots, J_i$ with weights w_{ij} such that

$$\sum_{j=1}^{J_i} w_{ij} = 1. \quad (5)$$

For each task j , let $s_{ij} \in [0, 1]$ denote substitution intensity and $c_{ij} \in [0, 1]$ denote augmentation (complementarity) intensity. Define occupation-level aggregate effects as

$$\alpha_i = \sum_{j=1}^{J_i} w_{ij} s_{ij}, \quad \beta_i = \sum_{j=1}^{J_i} w_{ij} c_{ij}. \quad (6)$$

Let $E_i(t)$ be employment demand for occupation i at time t , and let g_i capture baseline growth/decay from non-AI factors (estimated from public trends or set to 0 under scenario analysis). Given adoption trajectories $A(t) \in [0, 1]$ (fast/medium/slow), we model occupational evolution as

$$\frac{dE_i(t)}{dt} = E_i(t) \left(g_i + A(t) (\beta_i - \alpha_i) \right). \quad (7)$$

This formulation ensures occupational outcomes are derived from task-level mechanisms, supporting interpretability, reproducibility, and data-informed reasoning.

4.4 GenAI Capability-Dimension Scoring from Authoritative Reports: Scale Definition and Evidence Narrative

To replace subjective keyword tagging, we adopt capability ratings derived from authoritative evaluations and official technical reports. These ratings quantify GenAI **substitution** and **augmentation** capabilities across major software-development task dimensions, providing auditable grounds for mapping Software Developer Task DNA to (s, c) inputs.

4.4.1 Scale Definition

We use three levels (Low/Medium/High) to represent GenAI capability, encoded as 0.25, 0.50, and 0.75:

- **Low (0.25):** GenAI has limited autonomous ability on this task type and cannot reliably deliver satisfactory results without human intervention; it replaces only a small portion of work, but can assist under human supervision.

- **Medium (0.50):** GenAI performs at a moderate level, completing parts of the task or offering useful recommendations, yet still requires human review and collaboration; it cannot independently replace human execution.
- **High (0.75):** GenAI performs strongly and is close to professional human quality for many subtasks; under proper constraints and verification, it can handle a large share of the workflow and deliver significant augmentation, though it remains imperfect.

Note: the values 0.25/0.50/0.75 are coarse modeling encodings rather than precise measurements. All levels are grounded in publicly available reports and comparative experiments released within the last two years.

4.4.2 Capability Evidence by Task Type (Narrative Justification)

Code Generation **Substitution: High (0.75); Augmentation: High (0.75).** GenAI has achieved strong performance in automated code generation. OpenAI GPT-4 reaches 67.0% pass@1 on HumanEval, far above GPT-3.5 at 48.1%; Anthropic Claude 2 reaches 71.2% on the same benchmark. This indicates that leading models solve more than half of typical coding problems correctly. Google CEO Sundar Pichai stated that roughly 25% of new code at Google is generated by AI tools, reflecting substantial productivity gains: given intent or comments, models can produce large volumes of draft code and accelerate development. For simple modules, models can partially substitute manual coding. Nevertheless, outputs still require human review and testing to mitigate logic errors and security issues. Overall, coding/implementation is among the most substitution-prone and augmentation-rich domains for GenAI.

Testing and Verification **Substitution: Medium (0.50); Augmentation: High (0.75).** GenAI shows moderate but improving capability for generating test cases and supporting verification. Studies report that LLM-generated unit tests can achieve coverage comparable to advanced automated tools and are often more readable. For instance, IBM Research experiments guided by static analysis found that GPT-based systems can generate Java/Python unit tests with competitive coverage and defect detection while producing developer-friendly tests. This supports strong augmentation in routine function-level verification. However, generated tests may target irrelevant scenarios or weak assertions, requiring human review; thus full substitution remains difficult. In practice, GenAI can substantially reduce manual test writing and speed up result checking, while humans validate test correctness.

Debugging and Bug Fixing **Substitution: Low (0.25); Augmentation: Medium (0.50).** End-to-end debugging remains a weak area: autonomous success rates are relatively low, but assistant value is tangible. A 2025 Microsoft Research study reported that even with debugger tool access, the best model (Anthropic Claude 3.7 Sonnet) solved only 48.4% of 300 curated debugging tasks on average, while others were around 30%. This suggests GenAI cannot fully replace human debugging. Limitations include ineffective use of debugging signals, poor strategy selection, and lack of stepwise reasoning traces. Therefore substitution is rated low. Meanwhile, augmentation is evident: with logs or exception traces, models can propose useful fixes; “self-debugging” via few-shot demonstrations improves success by about 2%–12% in some settings. In practice, GenAI functions more as an assistant than an independent mechanic.

System Design / Architecture **Substitution: Low (0.25); Augmentation: Medium (0.50).** Architecture and system design demand global reasoning and responsible trade-offs; GenAI currently provides limited support. While no single authoritative benchmark directly measures system-design ability, models often produce plausible but incomplete plans, miss critical constraints, or rely on incorrect assumptions. Because system design lacks a single correct answer and requires accountable decision-making, autonomous substitution is rated low. Still, augmentation can be meaningful: models can draft design documents, summarize patterns, and brainstorm alternatives as a starting point for architects, improving discussion efficiency.

Requirements Analysis **Substitution: Low (0.25); Augmentation: Medium (0.50).** Requirements work requires deep domain context and ambiguity resolution. GenAI can assist with summarizing and structuring text, but cannot replace analysts. Although dedicated evaluations are limited, related tasks such as NL-to-formal specifications provide evidence: on Spider (NL-to-SQL), even with enhancement techniques, GPT-4 gains are only about 2%–3%, and complex queries remain challenging. This implies unreliable understanding for complex, ambiguous requirements, so substitution is low. However, augmentation is moderate: models can summarize lengthy documents, generate user-story templates, propose clarification questions, and accelerate organization of requirements under human supervision.

User Communication and Documentation **Substitution: Medium (0.50); Augmentation: High (0.75).** GenAI excels in language generation, strongly augmenting user communication and technical documentation. GPT-4 and Claude 2 can produce coherent long-form drafts and adhere to structured formats (e.g., Markdown/JSON), enabling rapid drafting of API docs, manuals, and user responses. Many routine user inquiries can be answered fluently and consistently. Yet accuracy and hallucination risks remain, especially for system-specific details, so fully replacing human communication is still risky; substitution is rated medium while augmentation is rated high.

4.5 Authoritative Evaluation Driven Relabeling of Task Impacts: From Keyword Tagging to Capability-Dimension Assignment

To improve the defensibility and reproducibility of task impact parameters (s_{ij}, c_{ij}) , we discard subjective keyword tagging and adopt an **authoritative evaluations** $\rightarrow$ **capability dimensions** $\rightarrow$ **semantic matching** $\rightarrow$ **piecewise assignment** relabeling workflow. The key idea is to extract substitution/augmentation levels for a set of *capability dimensions* (e.g., code generation, debugging, testing/verification, requirements analysis, system design, documentation/communication) from official evaluation reports, map each Software Developer task to 1–2 capability dimensions, and then assign (s_j, c_j) on a discrete grid.

Capability mapping and traceability requirement For each task j , define its mapped capability set dim_j (typically 1–2 dimensions for interpretability). Task-level substitution and augmentation are assigned on a discrete scale:

$$s_j, c_j \in \{0.25, 0.5, 0.75\}. \quad (8)$$

To meet the “data-informed” audit standard, each task must carry a **traceable chain**: (i) mapped capability dimension(s) dim_j , (ii) at least one authoritative source link, and (iii) a brief explanation

Table 3: GenAI capability-dimension rubric from authoritative evaluation reports (for mapping Software Developer tasks to *s/c*)

Task Type	Substitution Score	Augmentation Score	Main evidence and support
Code generation	High (0.75)	High (0.75)	GPT-4 achieves 67.0% pass@1 on HumanEval [?]. Claude 2 reaches 71.2% [?]. Google reports about 25% of new code is generated by AI [?].
Testing and validation	Medium (0.50)	High (0.75)	LLM-generated unit tests can reach competitive coverage and readability. Studies suggest higher-quality tests when guided by static analysis [?]. Human review is still required, so substitution remains moderate.
Debugging and bug fixing	Low (0.25)	Medium (0.50)	Microsoft Research reports that even with debugging tools, end-to-end debugging success is below 50% (best model 48.4%) [?]. Prompted repair suggestions help, and “self-debugging” improves accuracy by up to 12 percentage points [?].
System design	Low (0.25)	Medium (0.50)	Complex architecture requires integrated reasoning; current GenAI often yields partial solutions [?]. Direct quantitative benchmarks are limited, but models can assist with drafts and documentation to speed design discussions [?].
Requirements analysis	Low (0.25)	Medium (0.50)	Model gains on NL→spec tasks are limited (e.g., 2–3% on Spider) [?]. However, summarization and question generation can improve elicitation and analysis efficiency [?].
User communication and documentation	Medium (0.50)	High (0.75)	Claude 2 and related models generate long-form structured documentation and answer FAQs efficiently [?]. Accuracy remains a concern; critical content still needs human verification [?].

of the task-to-dimension matching logic. This ensures (s_j, c_j) are reviewable and not based on researcher intuition.

Semantic thresholds, anomaly reporting, and limited human overrides Because semantic matching can incur both false negatives and false positives, the matching threshold is a controllable decision parameter: too high may miss valid matches; too low may introduce mismatches. Therefore, relabeling outputs must include similarity scores and an anomaly report (e.g., low-similarity matches, conflicting multi-dimension assignments), and allow a small human override list for a few high-weight tasks with ambiguous semantics. This preserves overall reproducibility while preventing a small number of mislabels from biasing the occupation-level aggregates (α_i, β_i) .

4.6 Aggregating Task DNA into Occupation-Level Shock Parameters: Software Developers

We aggregate Software Developer Task DNA from task-level inputs (w_{ij}, s_{ij}, c_{ij}) into occupation-level shock parameters $(\alpha_i, \beta_i, \mu_i)$, which feed the evolution equation $\frac{dE_i(t)}{dt} = E_i(t)(g_i + A(t)(\beta_i - \alpha_i))$.

4.6.1 Top- N Availability and Aggregation

In practice, O*NET task ratings may be missing for some tasks. We therefore adopt a “Top- N_i usable tasks” strategy: for occupation i , we keep only tasks with sufficient ratings to construct weights and renormalize so that $\sum_{j=1}^{N_i} w_{ij} = 1$. Occupation-level substitution pressure, augmentation gain, and net direction are defined as

$$\alpha_i = \sum_{j=1}^{N_i} w_{ij} s_{ij}, \quad \beta_i = \sum_{j=1}^{N_i} w_{ij} c_{ij}, \quad \mu_i = \beta_i - \alpha_i. \quad (9)$$

4.6.2 Interpretability Decomposition: Top- K Contribution Lower Bounds

For interpretability, we decompose contributions from the top-weight tasks. Define partial sums

$$\alpha_i^{(K)} = \sum_{j \in \text{Top-}K} w_{ij} s_{ij}, \quad \beta_i^{(K)} = \sum_{j \in \text{Top-}K} w_{ij} c_{ij}. \quad (10)$$

For the current Top-5 sample, $\sum_{j \in \text{Top-}5} w_{ij} = 0.360897$, with partial contributions $\alpha_i^{(5)} = 0.1804485$ and $\beta_i^{(5)} = 0.2153186$, indicating that high-weight tasks (often testing/maintenance/requirements/-collaboration) dominate the aggregate shocks.

4.6.3 Validity Check of the Scoring Rule: Avoiding Near-Constant s/c

If many tasks share the same discrete score (e.g., 0.50), then α_i and β_i become weakly discriminative, undermining mechanistic explanation. We therefore check weight-mass concentration over the discrete grid $\mathcal{G} = \{0, 0.25, 0.5, 0.75, 1\}$:

$$p_s(g) = \sum_{j: s_{ij}=g} w_{ij}, \quad p_c(g) = \sum_{j: c_{ij}=g} w_{ij}, \quad g \in \mathcal{G}, \quad (11)$$

using concentration and normalized entropy:

$$C_s = \max_{g \in \mathcal{G}} p_s(g), \quad C_c = \max_{g \in \mathcal{G}} p_c(g), \quad (12)$$

$$H_s = -\frac{\sum_{g \in \mathcal{G}} p_s(g) \ln p_s(g)}{\ln |\mathcal{G}|}, \quad H_c = -\frac{\sum_{g \in \mathcal{G}} p_c(g) \ln p_c(g)}{\ln |\mathcal{G}|}. \quad (13)$$

High C_s or low H_s flags score compression and triggers a minimal calibration.

4.6.4 Minimal Calibration: Category Defaults with Limited Manual Review

To avoid re-collecting data, we calibrate only s/c via a small set of task categories while keeping the task list and weights w_{ij} unchanged. Let $\kappa(j)$ map each task to a coarse category (e.g., requirements/stakeholders, coding/implementation, testing/validation/security, monitoring/operations) with a default (s, c) pair. We then manually review only a few top-weight tasks and record brief evidence notes, improving identifiability of (α_i, β_i) while preserving reproducibility and transparency.

5 Mechanistic Power Check and Occupation-Level Aggregation After Authoritative Relabeling (Software Developer)

After upgrading (s_j, c_j) from subjective keyword tagging to an **authoritative evaluations** $\rightarrow$ **capability dimensions** $\rightarrow$ **semantic matching** $\rightarrow$ **piecewise assignment** pipeline, we must validate **mechanistic power**: task-level heterogeneity in (s_j, c_j) should be sufficient, and task weights w_j should not collapse (α, β) into a single default-driven outcome. Otherwise, even authoritative sources would yield weak identifiability in the evolution model.

5.1 Occupation-Level Shock Aggregates

For the usable task set (size N , allowing $N < 20$) renormalized such that $\sum_{j=1}^N w_j = 1$, we define

$$\alpha = \sum_{j=1}^N w_j s_j, \quad \beta = \sum_{j=1}^N w_j c_j, \quad \mu = \beta - \alpha. \quad (14)$$

5.2 Diagnostics: Heterogeneity and Effective Diversity

(1) Weighted variance (heterogeneity)

$$\text{Var}_w(s) = \sum_{j=1}^N w_j (s_j - \alpha)^2, \quad \text{Var}_w(c) = \sum_{j=1}^N w_j (c_j - \beta)^2. \quad (15)$$

Low variance indicates compressed scoring or overly convergent mapping rules, weakening identifiability.

(2) Weighted tier histograms (avoid single-tier dominance) Let $\mathcal{G}_{LMH} = \{0.25, 0.5, 0.75\}$. Define tier weight mass

$$h_s(g) = \sum_{j: s_j=g} w_j, \quad h_c(g) = \sum_{j: c_j=g} w_j, \quad g \in \mathcal{G}_{LMH}. \quad (16)$$

Large $\max_g h_s(g)$ or $\max_g h_c(g)$ flags default-driven collapse.

(3) Weight mass by capability dimension (avoid weight concentration) For each task j , let dim_j be its mapped dimension set (1–2 dimensions). If multi-mapped, let π_{jd} allocate task j to dimension d with $\sum_{d \in \text{dim}_j} \pi_{jd} = 1$. Dimension weight mass is

$$m(d) = \sum_{j=1}^N w_j \pi_{jd}. \quad (17)$$

High $\max_d m(d)$ indicates low effective diversity despite broad nominal coverage.

5.3 Engineering Thresholds and Actions (Gatekeeping Before $E(t)$)

We use engineering thresholds as a gate to decide whether to proceed to scenario evolution:

$$\text{Var}_w(s) \geq \varepsilon_s, \quad \text{Var}_w(c) \geq \varepsilon_c, \quad \max_g h_s(g) \leq \eta_s, \quad \max_d m(d) \leq \eta_d, \quad (18)$$

with defaults $\varepsilon_s = \varepsilon_c = 0.005$, $\eta_s = 0.70$, and $\eta_d = 0.60$. Violations imply high mechanism risk, requiring refinement of tiering or the dimension-to- (s, c) mapping (without changing w_j) before advancing to scenario-driven $E(t)$ dynamics.

6 Consistency Check and Instantiating the Evolution Equation: (α, β, μ) for Software Developers

After relabeling task impacts via **authoritative evaluations** $\rightarrow$ **capability dimensions** $\rightarrow$ **semantic matching** $\rightarrow$ **discrete tiers** (s, c) , we aggregate Task DNA back to occupation-level shock parameters (α, β, μ) and conduct a consistency check first, preventing downstream dynamics from relying on an incorrectly implemented net-shock term.

6.1 Aggregates and Consistency Check

By definition,

$$\mu = \beta - \alpha. \quad (19)$$

Given the preview values $\alpha = 0.379113$ and $\beta = 0.629112$, we must have

$$\mu = 0.629112 - 0.379113 = 0.249999. \quad (20)$$

However, the preview JSON reports $\mu = 0.504112$, which contradicts the model definition. Therefore, before using μ as an input to $E(t)$, the aggregation output must explicitly state `mu_definition` and correct the implementation; otherwise, the evolution inference becomes systematically biased.

6.2 Mechanistic Power (Heterogeneity) Check: No Score Collapse

Using weighted variances

$$\text{Var}_w(s) = \sum_{j=1}^N w_j (s_j - \alpha)^2, \quad \text{Var}_w(c) = \sum_{j=1}^N w_j (c_j - \beta)^2, \quad (21)$$

the preview reports $\text{Var}_w(s) = \text{Var}_w(c) = 0.015608$, exceeding an engineering threshold (e.g., 0.005). This indicates that the relabeled (s, c) retain sufficient heterogeneity; mechanistic identifiability is not collapsed.

6.3 Minimal Sensitivity for a Low-Confidence Task (Upper-Bound Argument)

For a low-confidence match (e.g., task_id=21666 with weight $w = 0.05434$), robustness can be justified with a worst-case bound. Since tiers are $\{0.25, 0.5, 0.75\}$, the maximum per-task tier swing is 0.50 ($0.25 \leftrightarrow 0.75$), hence

$$\Delta_{\max}(\alpha) \leq w \cdot 0.50 = 0.05434 \times 0.50 = 0.02717, \quad (22)$$

and similarly $\Delta_{\max}(\beta) \leq 0.02717$. Two lightweight scenarios (dropping the task; remapping to the second-best dimension) suffice for a defensible sensitivity statement.

6.4 Proceeding to Occupational Evolution After Fixing μ

Once μ is corrected, the evolution equation becomes

$$\frac{dE(t)}{dt} = E(t)(g + A(t)\mu), \quad (23)$$

where $A(t)$ is the adoption scenario (fast/medium/slow) and g is the non-AI baseline (public trend estimate or set to 0 under a relative-index convention). The relative scale has the closed-form expression

$$\frac{E(t)}{E(t_0)} = \exp \left(\int_{t_0}^t (g + A(\tau)\mu) d\tau \right). \quad (24)$$

Accordingly, we do not advance to $E(t)$ scenario curves until μ is definition-consistent and $A(t)/g$ are clearly specified, avoiding non-computable narratives.

7 Task-Level Functional Demand Model and Occupational Re-definition

7.1 Task-Level Functional Demand Model (Replacing Occupation-Level Job Counts)

7.1.1 Modeling Objective and Notation

We do not predict occupation-level job counts; instead, we predict how the *task composition* of an occupation evolves under GenAI adoption and how the *human labor demand* associated with each

task changes over time. The core claim is that when a sufficiently large portion of an occupation's baseline task bundle becomes negligible, the nature of the occupation has changed and should be redefined (the exact new title is not the core of the model).

Consider an occupation i decomposed into tasks $j = 1, \dots, J_i$. For each task, we assume the following inputs have already been constructed through Task DNA and the authoritative relabeling pipeline:

- Baseline task weights $w_{ij}(0)$ with $\sum_j w_{ij}(0) = 1$.
- Substitution score $s_{ij} \in [0, 1]$: the extent to which GenAI reduces human effort for task j .
- Augmentation score $c_{ij} \in [0, 1]$: the extent to which GenAI expands the *functional* scope/demand associated with task j .
- Adoption curve $A(t) \in [0, 1]$: GenAI adoption share/intensity at time t (scenario-driven).

We introduce four state variables:

- $Q_i(t)$: overall functional output demand index for occupation i ,
- $\ell_{ij}(t)$: labor intensity of task j (labor per unit output),
- $w_{ij}(t)$: time-varying task labor share (task structure),
- $D_{ij}(t)$: labor-demand index for task j .

Intuitively, s_{ij} governs how fast human effort becomes unnecessary (automation channel), while c_{ij} governs how much overall functional demand can expand (scaling channel). These are distinct mechanisms and are not forced to offset each other.

7.1.2 Two-Channel Dynamics: Automation and Scaling

(a) Automation channel: substitution reduces labor intensity. We assume that as GenAI adoption increases, tasks with higher substitution scores require less human effort per unit output:

$$\frac{d \ln \ell_{ij}(t)}{dt} = -\kappa_s A(t) s_{ij}, \quad \kappa_s > 0. \quad (25)$$

The closed-form solution is

$$\ell_{ij}(t) = \exp\left(-\kappa_s s_{ij} \int_0^t A(\tau) d\tau\right). \quad (26)$$

Parameter meaning (plain language). κ_s is the *substitution speed* coefficient. A larger κ_s implies faster decline in labor effort under the same $A(t)$ and s_{ij} .

(b) Task-structure update: internal redistribution within the occupation. Automation changes the internal composition of the occupation. We form an unnormalized share and renormalize:

$$\tilde{w}_{ij}(t) = w_{ij}(0) \ell_{ij}(t), \quad w_{ij}(t) = \frac{\tilde{w}_{ij}(t)}{\sum_k \tilde{w}_{ik}(t)}. \quad (27)$$

Interpretation. Highly automated tasks (small $\ell_{ij}(t)$) lose share; remaining labor shifts toward harder-to-substitute tasks.

(c) Scaling channel (optional): augmentation expands overall functional demand. To capture the possibility that productivity tools increase the total amount of work organizations choose to do, we model the overall functional demand index:

$$\frac{d \ln Q_i(t)}{dt} = g_i + \kappa_c A(t) \bar{c}_i, \quad (28)$$

where

$$\bar{c}_i = \sum_j w_{ij}(0) c_{ij}. \quad (29)$$

Parameter meaning (plain language). g_i is the baseline (non-GenAI) growth in functional demand; we can set $g_i = 0$ to avoid macro assumptions. $\kappa_c \geq 0$ is the *augmentation demand multiplier*. $\bar{c}_i$ summarizes the augmentation potential under the baseline composition. (A variant uses $w_{ij}(t)$ instead of $w_{ij}(0)$; we use (29) for simplicity and parameter stability.)

(d) Task labor-demand index. We allocate functional demand to tasks using the current task structure:

$$D_{ij}(t) = Q_i(t) w_{ij}(t), \quad \frac{D_{ij}(t)}{D_{ij}(0)} = \frac{Q_i(t) w_{ij}(t)}{w_{ij}(0)}. \quad (30)$$

Interpretation. Even if overall demand $Q_i(t)$ grows, a task's labor demand can still shrink if its share $w_{ij}(t)$ collapses due to substitution.

7.1.3 Adoption Scenarios $A(t)$

We use a simple and interpretable logistic family:

$$A(t) = \frac{1}{1 + \exp(-k(t - t_0))}. \quad (31)$$

Parameter meaning (plain language). k controls steepness (speed of acceleration). t_0 is the inflection time at which adoption reaches 50%. We define fast/medium/slow scenarios by choosing three (k, t_0) pairs; this keeps the number of drivers small while supporting sensitivity comparison.

7.1.4 Occupational Redefinition Triggers (Change in the Nature of Work)

We formalize “change in the nature of work” through *task extinction* and *task-structure drift*. Either criterion can trigger redefinition.

(a) Extinction mass (task demand becomes negligible). When a task's labor-demand index drops below a small fraction of its baseline, we treat the task as functionally negligible:

$$\frac{D_{ij}(t)}{D_{ij}(0)} < \varepsilon, \quad (32)$$

where $\varepsilon \in (0, 1)$ (e.g., $\varepsilon = 0.05$). Define the baseline-weighted extinction mass:

$$M_{\text{drop}}(t) = \sum_{j: D_{ij}(t)/D_{ij}(0) < \varepsilon} w_{ij}(0). \quad (33)$$

We say the occupation has fundamentally changed if

$$M_{\text{drop}}(t) \geq \theta, \quad (34)$$

where $\theta \in (0, 1)$ (e.g., $\theta = 0.30$) is the baseline task-share threshold that must become negligible before redefinition is triggered.

(b) Task-structure drift (distributional change). Even if few tasks become fully negligible, the occupation can change in nature if the task-share distribution shifts sufficiently. Let $w_i(t)$ denote the vector $(w_{i1}(t), \dots, w_{iJ_i}(t))$. We use the Jensen–Shannon divergence as a symmetric, bounded drift metric:

$$\text{JS}(w_i(t) \parallel w_i(0)) \geq \delta, \quad (35)$$

where $\delta > 0$ is a conservative threshold chosen for robustness.

Trigger time. Define

$$t^* = \inf \{t : M_{\text{drop}}(t) \geq \theta \text{ or } \text{JS}(w_i(t) \parallel w_i(0)) \geq \delta\}. \quad (36)$$

We report whether t^* exists within the forecast horizon and use it as model-based evidence that the occupation should be redefined.

7.1.5 Computation Algorithm (Reproducible Workflow)

For each occupation i and each adoption scenario:

1. Compute $A(t)$ from (31) and the cumulative adoption integral $I(t) = \int_0^t A(\tau) d\tau$ (numerical integration).
2. Compute labor intensity $\ell_{ij}(t)$ using (26).
3. Update task shares $w_{ij}(t)$ via (27).
4. Solve for $Q_i(t)$ using (28). If we avoid macro assumptions, set $g_i = 0$ and $\kappa_c = 0$, yielding $Q_i(t) \equiv 1$.
5. Compute task labor-demand indices $D_{ij}(t)$ from (30).
6. Evaluate redefinition criteria (34) and (35); if triggered, obtain t^* via (36).

7.1.6 Parameter Table (To Be Instantiated)

Table 4 summarizes the parameters to specify. We prioritize interpretability and a small number of drivers.

7.2 Visualization Plan

We define the plots needed to present task-structure change and redefinition evidence.

7.2.1 Figure Templates

Figure 1: Adoption scenarios $A(t)$.

Table 4: Task-level model parameters and plain-language meanings.

Parameter	Typical range/setting	Plain-language meaning
k, t_0	Scenario-specific	Adoption speed and 50% adoption time
κ_s	> 0	Speed at which substitution lowers labor effort
g_i	0 (default)	Baseline functional-demand growth (non-GenAI)
κ_c	0 or > 0	Strength of augmentation-driven demand expansion
ε	e.g., 0.05	“Negligible task” threshold (5% of baseline)
θ	e.g., 0.30	Extinction mass needed for redefinition
δ	e.g., 0.10–0.30	Task-share drift threshold (JS divergence)

Figure 4: GenAI adoption scenarios $A(t)$ defined by (31) (fast/medium/slow). The inflection time t_0 marks 50% adoption.

Figure 2: Task labor-demand trajectories for top tasks.

Figure 3: Task-structure drift $w_{ij}(t)$.Figure 4: Redefinition evidence and trigger time t^* .

7.3 Sensitivity and Robustness

7.3.1 Low-Confidence Task Sensitivity

Because semantic matching to authoritative dimensions can be weak for some tasks, certain tasks may receive low-confidence (s_{ij}, c_{ij}) assignments. For each flagged task, we run two bounded sensitivity cases:

- **Drop:** remove the task and renormalize remaining $w_{ij}(0)$.
- **Remap:** keep $w_{ij}(0)$ but replace (s_{ij}, c_{ij}) with the second-best acceptable mapping (or a conservative bound).

We report changes in (i) whether redefinition occurs within the horizon, (ii) trigger time t^* , and (iii) the extinction-mass curve $M_{\text{drop}}(t)$.

7.3.2 Key-Parameter Sensitivity

We vary only a minimal, high-leverage parameter set to avoid overfitting:

- κ_s (substitution speed): slow/medium/fast values.
- κ_c (scaling on/off): $\kappa_c = 0$ versus positive values.
- Thresholds $(\varepsilon, \theta, \delta)$: conservative vs. aggressive settings.

For each combination, we summarize qualitative stability (which tasks become negligible and whether/when redefinition is triggered).

Figure 5: Relative task labor-demand trajectories $D_{ij}(t)/D_{ij}(0)$ for the top- K baseline-weight tasks in occupation i across adoption scenarios. This figure shows which tasks become negligible and which remain core.

Figure 6: Time-varying task structure $w_{ij}(t)$ (stacked area or heat map). This figure visualizes internal reallocation driven by substitution in (27).

7.4 Output Summary Template (Per Occupation)

For each occupation and adoption scenario, we present:

1. **Task-demand results:** a ranked list by $D_{ij}(15)/D_{ij}(0)$, highlighting tasks below ε .
2. **Task-structure change:** visualization of $w_{ij}(t)$ and drift metric $JS(w_i(t)||w_i(0))$.
3. **Redefinition decision:** whether criteria in (34) or (35) are met, and trigger time t^* (if applicable).

8 Simulation Results and Sensitivity Analysis

We provide visual evidence through plots and discuss model robustness. Sensitivity analysis is crucial for achieving Meritorious or Outstanding awards.

9 Discussion: Strengths and Weaknesses

9.1 Strengths

High predictive accuracy; adaptable to high-dimensional data.

9.2 Weaknesses

Computational complexity for large-scale matrices.

10 Conclusion

We summarize findings and provide actionable recommendations.

References

Appendices

Appendix A Python Implementation Code

Figure 7: Redefinition evidence over time: extinction mass $M_{\text{drop}}(t)$ with threshold θ , and (optional) $\text{JS}(w_i(t) \| w_i(0))$ with threshold δ . The earliest crossing defines t^* in (36).

```
1 def model_solver(data):  
2     # Core mathematical logic  
3     return results
```

Report on Use of AI Tools

1. Dialogue Traceability

For each use, provide the exact **Prompt/Query** and the **raw AI Output** (or a representative excerpt). Omit full logs only for translation-only use; in that case, state the **tool name and version**.

- **Example—Prompt:** “Suggest an outline for the model section.” **Output (excerpt):** “1. State assumptions; 2. Define notation; 3. Derive main equations. . .”
- **Translation-only teams:** Declare tool name and version (e.g., DeepL 4.2, ChatGPT-4); no full prompt log required.

2. Phased Declaration

State in which phases AI was used, e.g.:

- **Initial conception / brainstorming**
- **Code correction / debugging**
- **Translation / copy-editing**

Mark each phase as used or not, and name the tool per phase.

3. Risk Verification

Describe how the team checked for **hallucinations** and **wrong citations**: e.g., manual fact-check of key claims, verification of references, cross-check of formulas and code. If only used for translation, state that no substantive technical content was delegated to AI.